

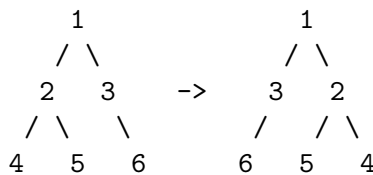
Part 10: Core LeetCode Patterns

Invert, subtree, LCA, construction, serialization, and just enough BST

These are the remaining patterns that appear over and over in Easy and Medium tree problems. Each one is short. What matters is recognising which one a new problem belongs to.

Invert a binary tree

LeetCode 226. Mirror the tree left to right.



```
TreeNode* invertTree(TreeNode* root) {
    if (root == NULL) return NULL;

    TreeNode* L = invertTree(root->left);
    TreeNode* R = invertTree(root->right);

    root->left = R;
    root->right = L;
    return root;
}
```

Read it as the standard template where the return value is “the fixed up version of my subtree”. Invert both sides, then swap.

You can also write it as: swap first, then recurse. Both work, because swapping and inverting are independent operations. What does **not** work is swapping and then recursing into `root->left` twice, which is easy to do by accident after the swap.

Time $O(N)$, space $O(H)$. A BFS version with a queue works equally well and is stack safe.

Merge two binary trees

LeetCode 617. Overlay two trees, adding values where both have a node.

```

TreeNode* mergeTrees(TreeNode* a, TreeNode* b) {
    if (a == NULL) return b;
    if (b == NULL) return a;

    a->val += b->val;
    a->left = mergeTrees(a->left, b->left);
    a->right = mergeTrees(a->right, b->right);
    return a;
}

```

The two base cases carry all the logic: if one side is missing, the answer is simply the other side, whole. This “walk two trees in step” shape is the same one used for Same Tree and Symmetric Tree in Part 5.

Subtree of another tree

LeetCode 572. Does root contain a subtree identical to subRoot?

Reuse `isSameTree` from Part 5, then try every node as a starting point.

```

bool isSameTree(TreeNode* p, TreeNode* q) {
    if (p == NULL && q == NULL) return true;
    if (p == NULL || q == NULL) return false;
    return p->val == q->val &&
        isSameTree(p->left, q->left) &&
        isSameTree(p->right, q->right);
}

bool isSubtree(TreeNode* root, TreeNode* subRoot) {
    if (root == NULL) return subRoot == NULL;
    if (isSameTree(root, subRoot)) return true;
    return isSubtree(root->left, subRoot) || isSubtree(root->right, subRoot);
}

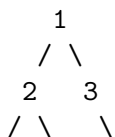
```

Time $O(N * M)$ in the worst case, where N and M are the two sizes. That is accepted for the given constraints. This is one of the rare cases where the “call a helper at every node” shape is the intended solution, because the helper stops early at the first mismatch.

Important distinction: a **subtree** must include a node and **all** of its descendants. A shape that appears in the middle of the tree but has extra children below it does not count. Students often confuse this with subtree matching in string problems.

Lowest Common Ancestor

LeetCode 236. Given two nodes p and q , find the deepest node that has both of them as descendants. A node counts as a descendant of itself.



4 5 6

LCA(4, 5) = 2

LCA(4, 6) = 1

LCA(2, 4) = 2 (a node can be its own ancestor)

The standard solution

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (root == NULL) return NULL;
    if (root == p || root == q) return root;

    TreeNode* L = lowestCommonAncestor(root->left, p, q);
    TreeNode* R = lowestCommonAncestor(root->right, p, q);

    if (L != NULL && R != NULL) return root;    // p and q are on opposite sides: I am the LCA
    return (L != NULL) ? L : R;                // both on one side, or neither found
}
```

The code is five lines and completely opaque until you state what the return value **means**:

$f(x)$ returns the LCA if both p and q are inside x 's subtree; otherwise it returns whichever of p or q it found; otherwise NULL.

With that sentence, every line is obvious:

- If I am p or q , I report myself and stop digging. (Correct even if the other node is below me, since then I am the LCA.)
- If the left returned something and the right returned something, p and q are in different subtrees, so I am the meeting point.
- Otherwise pass up whatever was found.

This assumes both p and q actually exist in the tree, which LeetCode 236 guarantees. If they might not exist, you need extra flags to distinguish “found both” from “found one”.

Time $O(N)$, space $O(H)$.

LCA in a BST

LeetCode 235. With the ordering promise you never have to search both sides.

```
TreeNode* lcaBST(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root) {
        if (p->val < root->val && q->val < root->val)    root = root->left;
        else if (p->val > root->val && q->val > root->val) root = root->right;
        else return root;                             // they split here
    }
    return NULL;
}
```

The first node where p and q fall on opposite sides (or where one equals the node) is the LCA. **$O(H)$ time, $O(1)$ space.**

The path based alternative

Find the root to p path and the root to q path using `findPath` from Part 9, then walk both from the front while they agree. The last agreeing node is the LCA. It is $O(N)$ time and $O(H)$ space, easier to explain, and it generalises to “distance between two nodes” (walk both paths, add the leftover lengths).

Build a tree from traversal arrays

Preorder plus inorder

LeetCode 105. The two facts you need:

- The **first** element of preorder is the root.
- Finding that value in inorder splits inorder into “everything on the left” and “everything on the right”.

preorder: [1, 2, 4, 5, 3, 6]

inorder : [4, 2, 5, 1, 3, 6]

root = 1

inorder left = [4, 2, 5] -> 3 nodes

inorder right = [3, 6]

preorder left = next 3 elements = [2, 4, 5]

preorder right= [3, 6]

Then recurse on each half. Searching for the root inside inorder every time costs $O(N)$ each, giving $O(N^2)$, so precompute a map from value to index.

```
class Solution {
    unordered_map<int,int> pos;    // value -> index in inorder
    int preIdx = 0;

    TreeNode* build(vector<int>& preorder, int inL, int inR) {
        if (inL > inR) return NULL;

        int val = preorder[preIdx++];
        TreeNode* root = new TreeNode(val);
        int mid = pos[val];

        root->left = build(preorder, inL, mid - 1);
        root->right = build(preorder, mid + 1, inR);
        return root;
    }

public:
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        pos.clear();
        for (int i = 0; i < (int)inorder.size(); i++) pos[inorder[i]] = i;
        preIdx = 0;
        return build(preorder, 0, (int)inorder.size() - 1);
    }
};
```

```
    }
};
```

The single global `preIdx` works because the left subtree is built completely before the right subtree starts, so preorder is consumed in exactly the right order. Build the left before the right, or everything breaks.

Time $O(N)$, space $O(N)$ for the map plus $O(H)$ recursion. This assumes distinct values, which the problem guarantees.

Postorder plus inorder

LeetCode 106. Same idea with two changes: the root is the **last** element of postorder, so consume postorder from the back, and build the **right** subtree before the left.

Why inorder alone is not enough

Inorder $[4, 2, 5]$ is produced by several different trees. Without knowing which node is the root you cannot split. Preorder alone is also insufficient in general. You need one order that identifies the root plus inorder that splits the rest. (Preorder plus postorder is also ambiguous for general binary trees.)

Serialize and deserialize

LeetCode 297. Turn a tree into a string and back.

The intuition: a traversal alone loses the shape, because you cannot tell missing children from present ones. Fix that by writing an explicit marker for every NULL. Preorder plus null markers determines the tree uniquely.

```
void ser(TreeNode* root, string& s) {
    if (root == NULL) { s += "#,"; return; }
    s += to_string(root->val) + ",";
    ser(root->left, s);
    ser(root->right, s);
}
```

Our tree becomes `1,2,4,#,#,5,#,#,3,#,6,#,#,.`

Rebuilding reads the tokens in the same preorder:

```
TreeNode* des(vector<string>& tok, int& i) {
    if (tok[i] == "#") { i++; return NULL; }
    TreeNode* root = new TreeNode(stoi(tok[i++]));
    root->left = des(tok, i);
    root->right = des(tok, i);
    return root;
}
```

`int& i` is passed by reference so all calls share one cursor, exactly like `preIdx` above. This is the same pattern twice, and it is worth naming: **one shared cursor walking a linear sequence while the recursion rebuilds the structure.**

Time $O(N)$, space $O(N)$. A BFS version with null markers works too and is what LeetCode's own display format uses.

Just enough BST

You do not need a full BST course to handle Easy and Medium tree problems, but these five come up often.

Search

$O(H)$ instead of $O(N)$, because the ordering tells you which side to enter. See Part 5.

Insert

LeetCode 701.

```
TreeNode* insertIntoBST(TreeNode* root, int val) {
    if (root == NULL) return new TreeNode(val);
    if (val < root->val) root->left = insertIntoBST(root->left, val);
    else                root->right = insertIntoBST(root->right, val);
    return root;
}
```

The `root->left = f(root->left)` idiom is worth noticing. It handles “create a new node here” and “nothing changed” with the same line. Deletion in a BST uses the same idiom and is genuinely fiddly, so leave it for later.

Validate a BST

LeetCode 98. The local check `left->val < root->val < right->val` is wrong, as shown in Part 1. Two correct approaches:

Range approach, information flowing down:

```
bool valid(TreeNode* root, long long lo, long long hi) {
    if (root == NULL) return true;
    if (root->val <= lo || root->val >= hi) return false;
    return valid(root->left, lo, root->val) &&
           valid(root->right, root->val, hi);
}
// call valid(root, LLONG_MIN, LLONG_MAX)
```

Every node narrows the allowed range for its children. Use `long long` bounds so that a node holding `INT_MIN` or `INT_MAX` does not break the comparison.

Inorder approach, using the sorted property: walk inorder and check that each value is strictly greater than the previous one. Keep a `TreeNode* prev` outside the recursion.

Kth smallest

LeetCode 230. Inorder with a counter, returning as soon as the counter hits k . $O(H + k)$, better than building the whole list.

Sorted array to balanced BST

LeetCode 108. Pick the middle element as the root, recurse on both halves. The middle choice is what guarantees balance.

```
TreeNode* build(vector<int>& a, int l, int r) {
    if (l > r) return NULL;
    int mid = l + (r - l) / 2;
    TreeNode* root = new TreeNode(a[mid]);
    root->left = build(a, l, mid - 1);
    root->right = build(a, mid + 1, r);
    return root;
}
```

Time $O(N)$, space $O(\log N)$.

Common mistakes

- **Recurring into `root->left` after already swapping it** in invert.
- **Confusing subtree with subgraph.** A subtree carries all descendants.
- **Explaining LCA code without stating what the function returns.** In an interview, state the meaning first, then the code.
- **Building the right subtree before the left in problem 105.** The shared preorder cursor makes order mandatory.
- **Searching inorder linearly for the root.** $O(N^2)$. Use a map.
- **Serializing without null markers.** The shape is lost and cannot be recovered.
- **Passing the cursor by value in deserialize.** Every call gets its own copy and the tree comes out wrong.
- **Using `INT_MIN` and `INT_MAX` as the initial range in validate BST.** Fails when a node actually holds those values. Use `long long`.
- **Assuming the tree is a BST when the problem never said so.**

Check yourself

1. In one sentence, what does the LCA function return for an arbitrary node?
2. Why does building the tree from preorder and inorder require a map, and what is the complexity without it?
3. Why is inorder alone not enough to rebuild a tree?
4. Why must the deserialize cursor be shared across calls?
5. What is wrong with checking only `left->val < root->val < right->val` for a BST?
6. Why does choosing the middle element in problem 108 give a balanced tree?
7. Subtree of another tree runs in $O(N*M)$. Why is that acceptable here, whereas an $O(N^2)$ diameter solution is not?